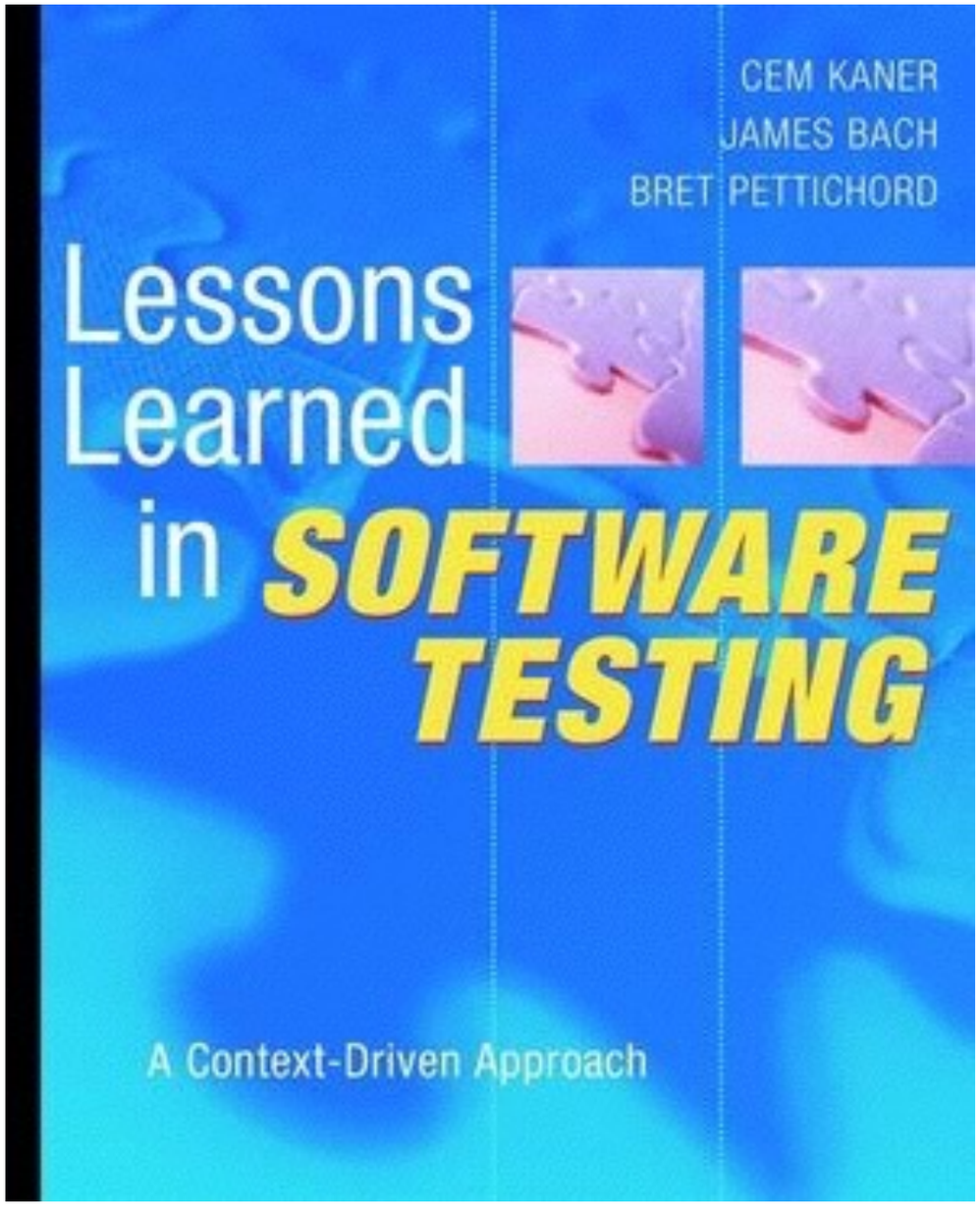
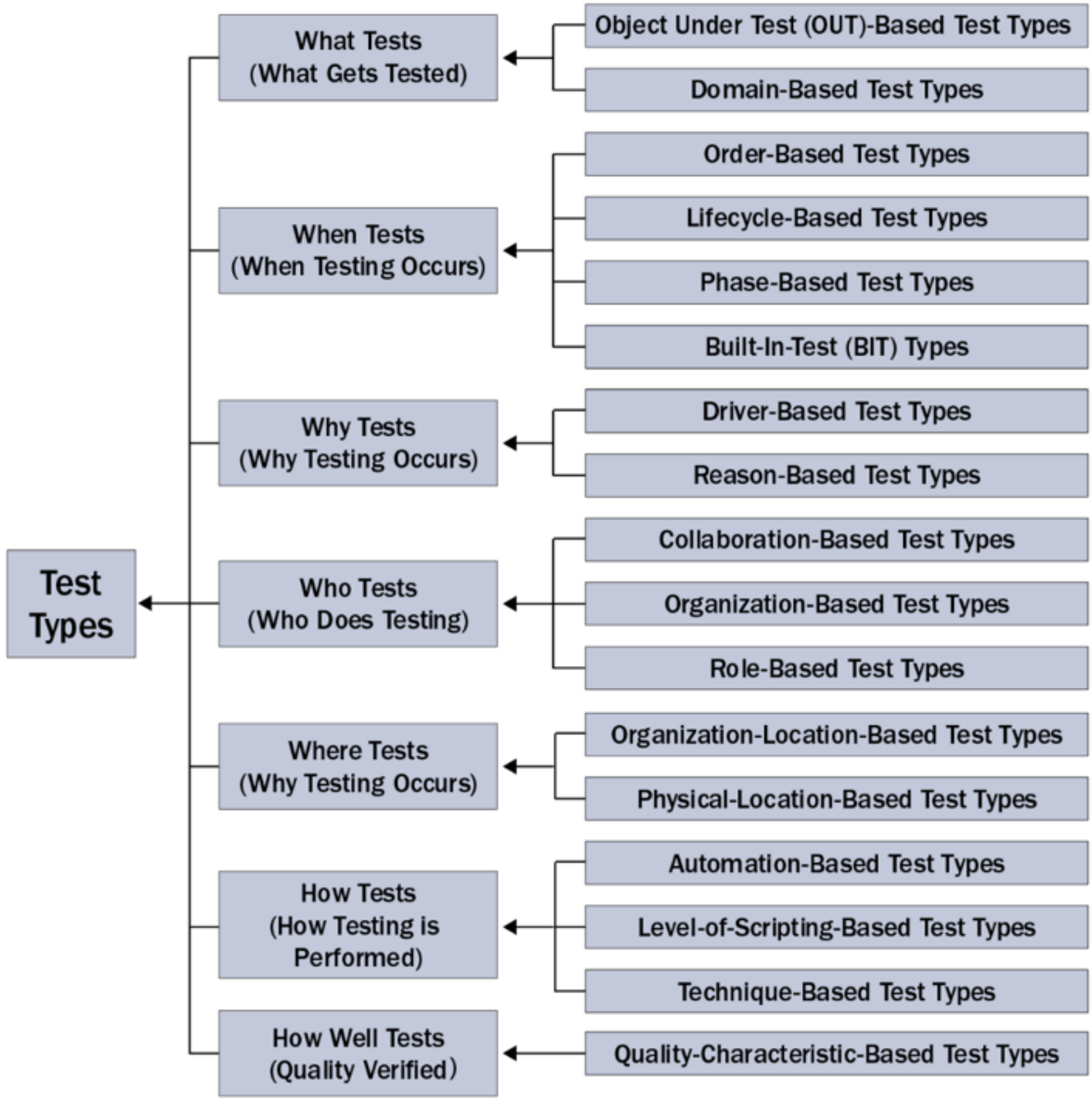
r7k

Testing Patience

Michael Feathers
R7K Research & Conveyance


CEM KANER
JAMES BACH
BRET PETTICHORD
Lessons Learned in SOFTWARE TESTING
A Context-Driven Approach


Test Types
What Tests (What Gets Tested)
Object Under Test (OUT)-Based Test Types
Domain-Based Test Types
When Tests (When Testing Occurs)
Order-Based Test Types
Lifecycle-Based Test Types
Phase-Based Test Types
Built-In-Test (BIT) Types
Why Tests (Why Testing Occurs)
Driver-Based Test Types
Reason-Based Test Types
Who Tests (Who Does Testing)
Collaboration-Based Test Types
Organization-Based Test Types
Role-Based Test Types
Where Tests (Why Testing Occurs)
Organization-Location-Based Test Types
Physical-Location-Based Test Types
How Tests (How Testing is Performed)
Automation-Based Test Types
Level-of-Scripting-Based Test Types
Technique-Based Test Types
How Well Tests (Quality Verified)
Quality-Characteristic-Based Test Types

The Goal - Fewer Errors in Production

The Goal - Fewer Errors in Production

Is that all?

Goal - Quality

June 12, 2008

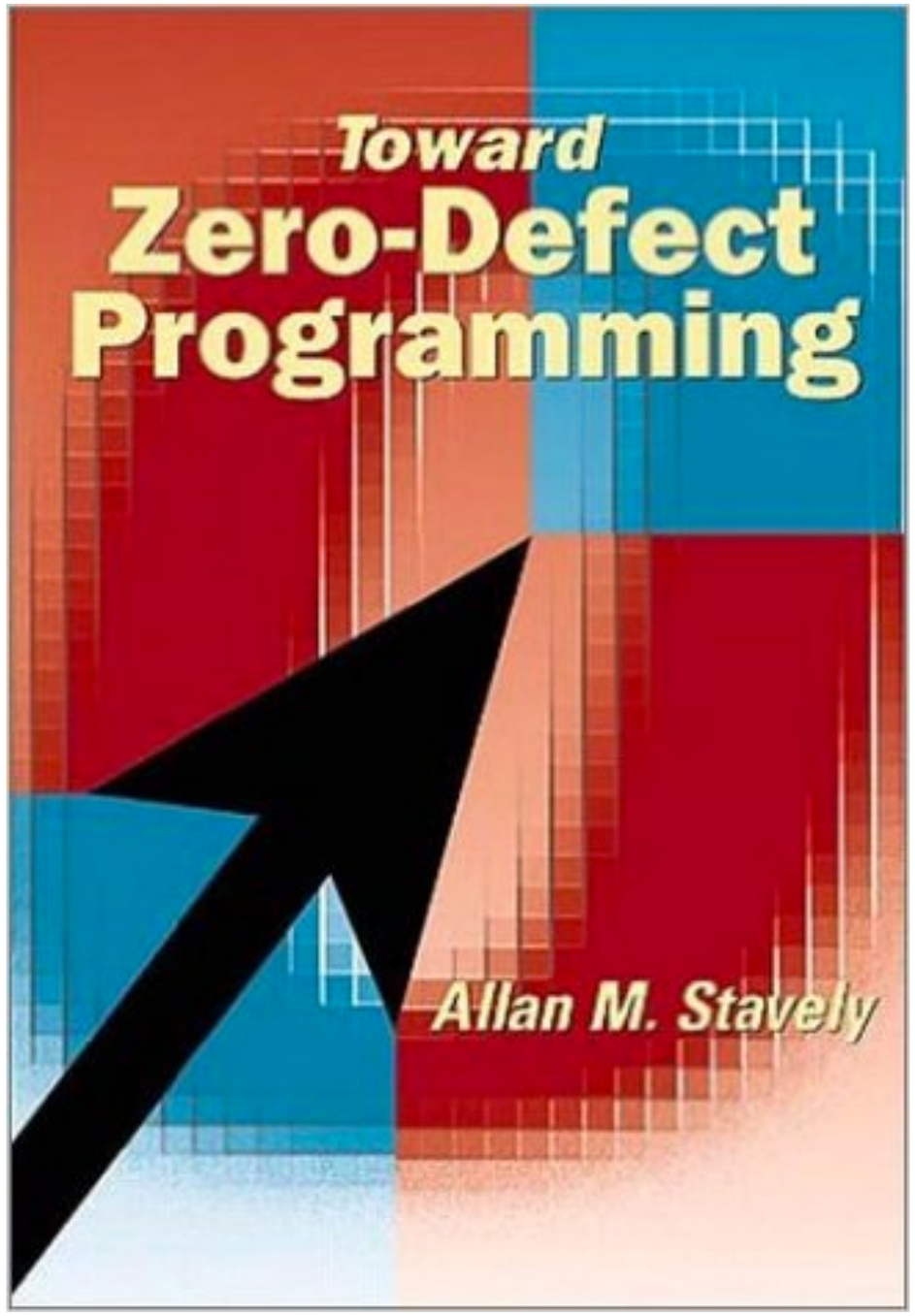
The Flawed Theory Behind Unit Testing

I have Google's blogsearch set to give me notifications about unit testing. On an average week, I read dozens of blogs and mailing list discussions about the topic. Occasionally, I read something new, but there's lot of repetition out there. The same arguments crop up often. Of all of them, though, there is one argument about unit testing which really bugs me because it rests on a flawed theory about testing and quality and sadly, it's an argument that I fell for a long time ago and I'd like to lay it to rest. Hopefully, this blog will help, but I have to relate a little history first.

Back in the very early 2000s, I had a conversation with Steve Freeman at a conference. We were talking about Test-Driven Development and Steve had the strong feeling that most of the people who were practicing TDD at the time were doing it wrong - they'd missed something.

Steve was and is part of a close-knit community in London who have been practicing XP and TDD from the very beginning. Among the fruits of their labor was the entire notion of mock objects. Steve Freeman and Tim MacKinnon wrote the paper that introduced the idea to the broader community. The rest is history. There are mock object frameworks out there for nearly every language in common use.

Quality is a function of precise thought and reflection


Toward
Zero-Defect
Programming
Allan M. Stavely

"A NASA satellite-control system. This was 40,000 lines of FORTRAN. The bug density in testing was 4.5 per thousand lines. In the first three years of production use, only seven minor errors were found, which is less than 0.1 per thousand lines, and each of these was easy to fix."

– Allan M. Stavely
(Toward Zero Defect Programming)

```
[y > x -> x, y, := y, x | true -> I]
if (y > x)
    [x, y := y, x]
    {
        int temp;
        temp = x;
        x = y;
        y = temp;
    }
```

"When are the intended functions written? They can be written after the code is written. If you did this, you would use the intended function for each construct to document what you had in mind when you wrote it .. Sometimes you might need to look at the construct itself, derive the function that it actually computes and use that as the "intended" function."

"Experienced programmers would probably do things in the opposite order most of the time."

– Allan M. Stavely

"When are the intended functions written? They can be written after the code is written. If you did this, you would use the intended function for each construct to document what you had in mind when you wrote it .. Sometimes you might need to look at the construct itself, derive the function that it actually computes and use that as the "intended" function."

"Experienced programmers would probably do things in the opposite order most of the time."

– Allan M. Stavely

TDD?

Quality Comes From Deliberate Thought

Quality Comes From Deliberate Thought

We can introduce constraints for force thought

Property-Based Testing

Jessica Kerr
@jessitron

Follow

Writing tests first forces you to think about the problem you're solving. Writing property-based tests forces you to think way harder.

1:52 AM - 26 Apr 2013

15 10

```
let ``append minValue then sort should be same as sort then prepend minValue`` sort
Fn aList =
    let minValue = Int32.MinValue

    let appendThenSort = (aList @ [minValue]) |> sortFn
    let sortThenPrepend = minValue :: (aList |> sortFn)
    appendThenSort = sortThenPrepend

// test
Check.Quick (``append minValue then sort should be same as sort then prepend minVal
ue`` goodSort)
// Ok, passed 100 tests.
```

(F# For Fun and Profit)

Property-Based Testing

examples $\longrightarrow$ $\forall$

Tactic 2:

Introducing Hazard to Make Systems Robust

Why making streets risky improves road safety

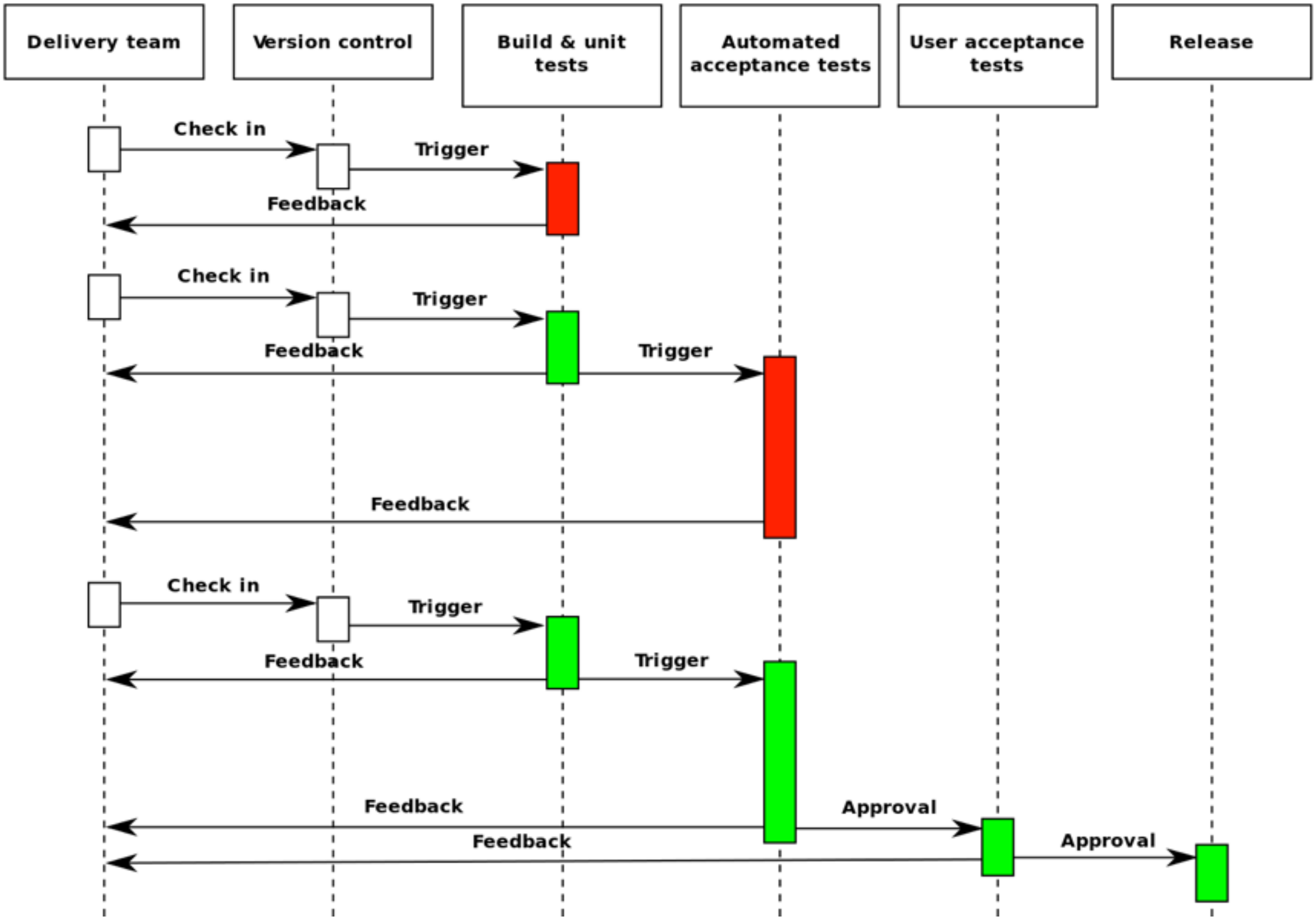
Mixing cars, cyclists and pedestrians may appear more dangerous, but 'Shared Space' approaches are reducing accidents across Europe.

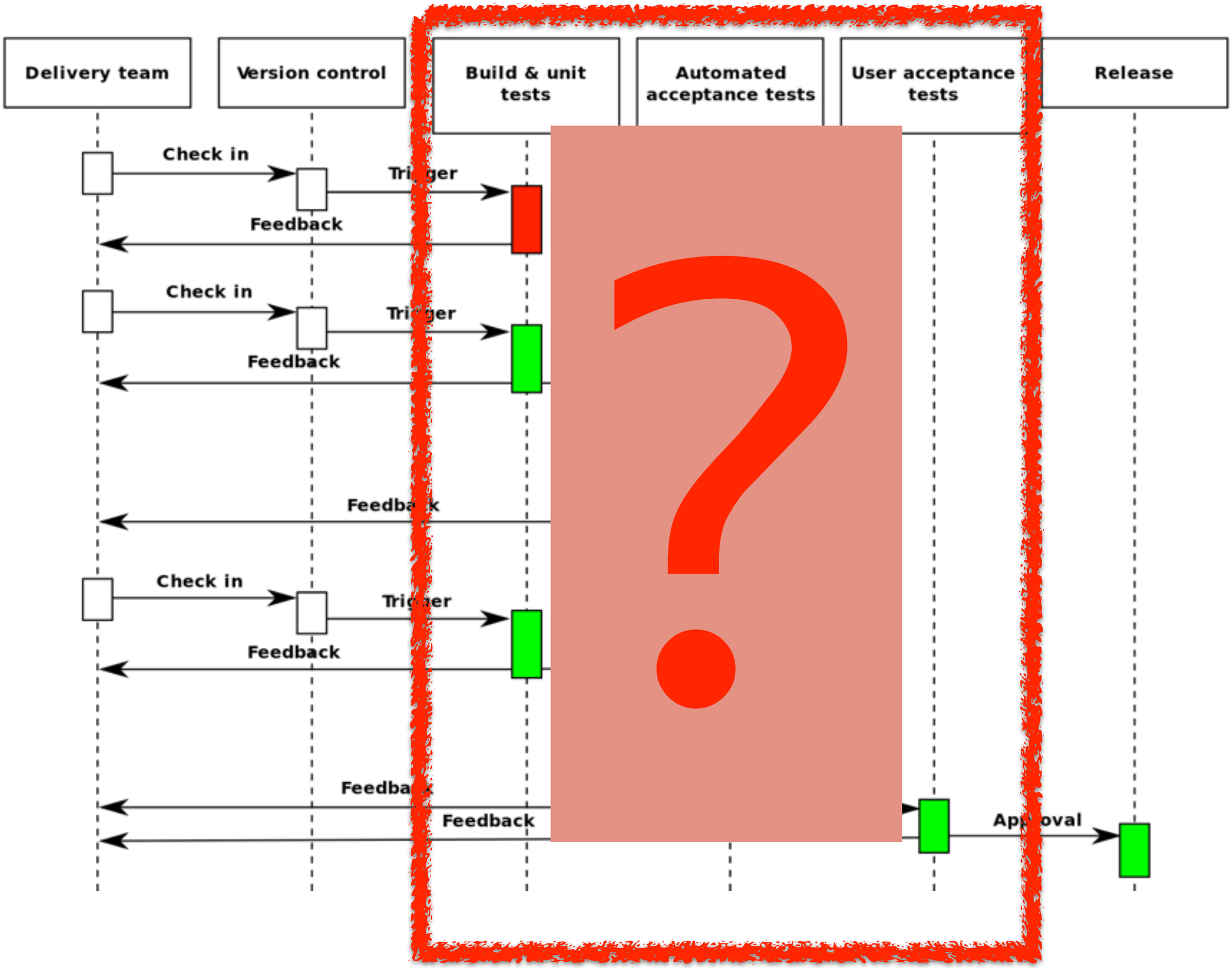
A study of seven European Shared Space projects from 2004 to 2008, backed by the European Interreg IIIB North Sea Programme, supports Hamilton-Baillie's view. "There are fewer accidents," it concludes. "When a situation feels unsafe, people are more alert."

Unexpectedly, the study also found that despite motorists driving more slowly in Shared Space zones traffic delays were reduced - by up to 50% at the Laweiplein, which sees 22,000 vehicles pass through each day.

Traffic lights, road marking and pedestrian crossings have also been removed in Ashford in England - which has seen "a 60% drop in accidents in the first three years", says Hamilton-Baillie.

Putting it into Production


Delivery team
Version control
Build & unit tests
Automated acceptance tests
User acceptance tests
Release
Check in
Trigger
Feedback
Check in
Trigger
Feedback
Trigger
Feedback
Check in
Trigger
Feedback
Trigger
Feedback
Approval
Feedback
Approval

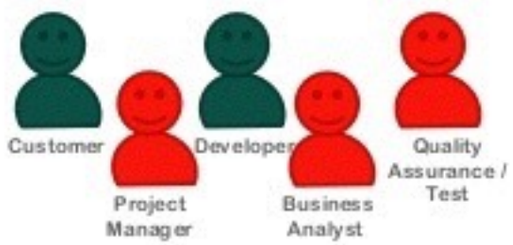

Delivery team
Version control
Build & unit tests
Automated acceptance tests
User acceptance tests
Release
Check in
Trigger
Feedback
Check in
Trigger
Feedback
Feedback
Check in
Trigger
Feedback
Feedback
Feedback
Approval

Tended Systems v. Untended Systems

26 Programmer Anarchy

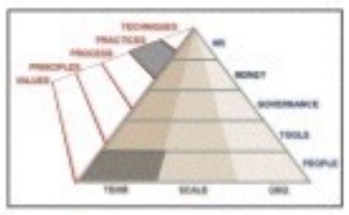
"Developer driven development"

Reduction in Roles



Missing Agile Practices

Agile Manifesto & XP Values
~~Standups~~ Trust with co location
~~Story narratives~~
~~Retrospectives~~
~~Estimates~~
~~Iterations~~ Results, not blame
~~Mandatory~~ pairing
~~Unit tests, acceptance tests~~
~~Refactoring~~
~~Patterns~~ Small, short lived apps
~~Continuous integration~~ Continuous deployment



Lack of managers
Lightweight (beyond Agile)
Works well for some companies

+

Limited reference material
Requires highly skilled / disciplined team
No testing or planning

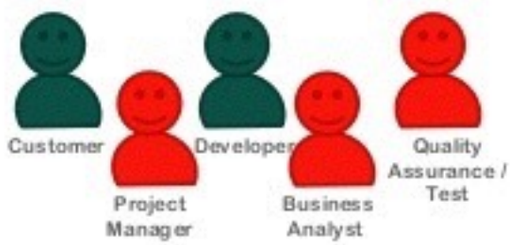
-

Fred George 2010 db.tt/4qFIjVxU

26 Programmer Anarchy

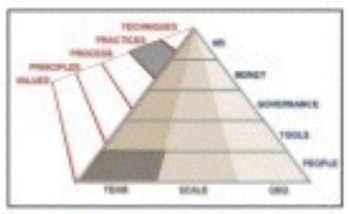
"Developer driven development"

Reduction in Roles



Missing Agile Practices

Agile Manifesto & XP Values
~~Standups~~ Trust with co location
~~Story narratives~~
~~Retrospectives~~
~~Estimates~~
~~Iterations~~ Results, not blame
~~Mandatory~~ pairing
~~Unit tests, acceptance tests~~
~~Refactoring~~
~~Patterns~~ Small, short lived apps
~~Continuous integration~~ Continuous deployment



Lack of managers
Lightweight (beyond Agile)
Works well for some companies

+

Limited reference material
Requires highly skilled / disciplined team
No testing or planning

-

Fred George 2010 db.tt/4qFIjVxU

If you set the context, you can get rid of many constraints

Moral hazard is a situation in which one party gets involved in a risky event knowing that it is protected against the risk and the other party will incur the cost.

Moral hazard is a situation in which one party gets involved in a risky event knowing that it is protected against the risk and the other party will incur the cost.

..having a QA Department

Death/Repurposing of QA

Goal - Maintenance

The Golden Master approach

Before making any change to the production code, do the following:

1. Create X number of random inputs, always using the same random seed, so you can generate always the same set over and over again. You will probably want a few thousand random inputs.
2. Bombard the class or system under test with these random inputs.
3. Capture the outputs for each individual random input

(from Sandro Mancuso)

https://dzone.com/articles/testing-legacy-code-golden

Goal - Validation

Verification and Validation

The PMBOK guide, a standard adopted by IEEE, defines them as follows in its 4th edition:[2]

- "Validation. The assurance that a product, service, or system meets the needs of the customer and other identified stakeholders. It often involves acceptance and suitability with external customers. Contrast with *verification*."
- "Verification. The evaluation of whether or not a product, service, or system complies with a regulation, requirement, specification, or imposed condition. It is often an internal process. Contrast with *validation*."

Verification and Validation

The PMBOK guide, a standard adopted by IEEE, defines them as follows in its 4th edition:[2]

- "Validation. The assurance that a product, service, or system meets the needs of the customer and other identified stakeholders. It often involves acceptance and suitability with external customers. Contrast with *verification*."
- "Verification. The evaluation of whether or not a product, service, or system complies with a regulation, requirement, specification, or imposed condition. It is often an internal process. Contrast with *validation*."

Much development is moving toward a post-verification world

Testing as a thinking tool - property based behavioral invariant verification

Quality
Maintenance
Validation

Force Thinking - Quality
Behavioral Invariants - Maintenance
Acceptance - Validation

Force Thinking - Quality
Behavioral Invariants - Maintenance
Acceptance - Validation

Moving toward Transience

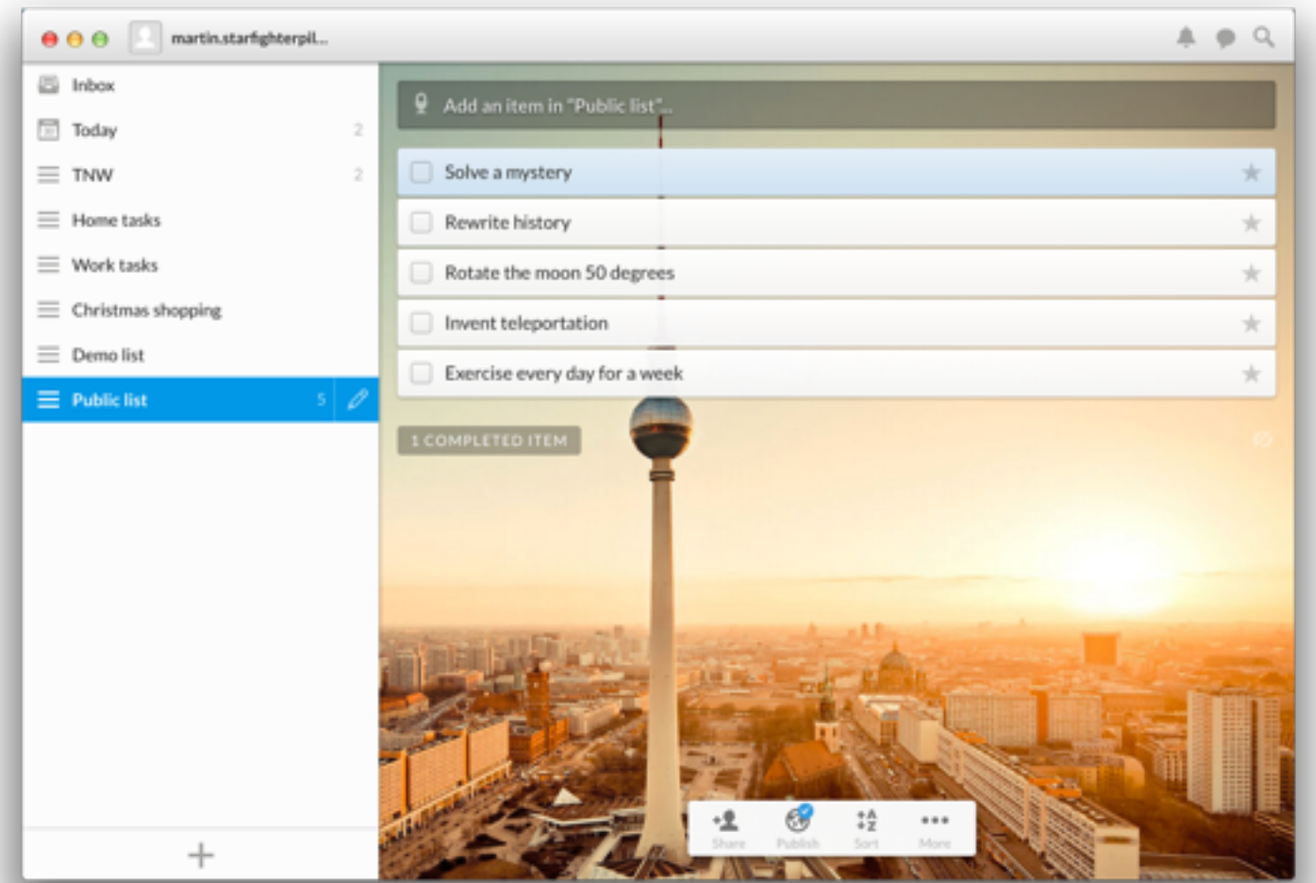
Twitter Inc.

Company

twitter.com

Twitter is an online social networking service that enables users to send and read short 140-character messages called "tweets". Registered users can read and post tweets, but those who are unregistered can only read them. Wikipedia

Stock price: TWTR (NYSE) $14.39 -0.22 (-1.54%)



Rewrite services when you want to

```
intervals
  .sort
  .map { |interval| [interval.first, intervals.reject { |other| (other <=> interval) == 1 }.flatten.max] }
  .slice_when {|c,n| c.last < n.first }
  .map {|group| group.flatten.minmax }
```

The Goal - Less Badness

r7k

michael.feathers@r7krecon.com
r7krecon.com